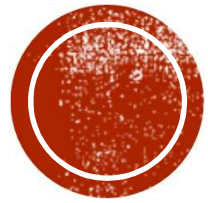


CME 3206

SOFTWARE ENGINEERING

Assoc. Prof. Dr. Kokten Ulas BIRANT





WEEK 6. TESTING



VERIFICATION

- Verification refers to the set of activities that ensure that software correctly implements a specific function.
- “Are we building the product right?”

*Verification: Doğrulama

VALIDATION

- Validation refers to a different set of activities that ensure that the software that has been built is traceable to customer requirements.
- “Are we building the right product?”

*Validation: Onaylama



TESTING

- Software must be tested to uncover (and correct) as many errors as possible before delivery to your customer.
- The engineer creates a series of test cases that are intended to “demolish” the software that has been build.

TESTING OBJECTIVES

- Testing is a process of executing a program with the intent of finding an error.
- A good test case is one that has a high probability of finding an as-yet-undiscovered error.
- A successful test is one that uncovers an as-yet-undiscovered error.

*intent: Amaç

TESTING PRINCIPLES

- All tests should be traceable to customer requirements.
- Tests should be planned long before testing begins.
- The Pareto (%80->%20) principle applies to software testing.
- Testing should begin “in the small” and progress toward testing “in the large”.
- Exhaustive testing is not possible.
- To be most effective, testing should be conducted by an independent third party.

*Exhaustive: Geniş, detaylı

TESTABILITY (1/2)

- SW Testability is simply how easily can be tested.
- Metrics for testability:
 - Operability (The better it works, the more efficiently it can be tested)
 - Observability (What you see is what you test)
 - Controllability (The better we can control the software, the more the testing can be automated and optimized)



TESTABILITY (2/2)

- Metrics (cont.):
 - Decomposability (By controlling the scope of testing, we can more quickly isolate problems and perform smarter retesting)
 - Simplicity (The less there is to test, the more quickly we can test it)
 - Stability (The fewer the changes, the fewer the disruptions to testing)
 - Understandability (The more information we have, the smarter we will test)

COMPLETION OF TESTING

- We cannot be absolutely certain that the software will never fail, but relative to a theoretically sound and experimentally validated statistical model, we have done sufficient testing to say with 95 percent confidence.

PROCESS STAGES

- 5 stages:
 - Unit Testing: Individual components are tested to ensure that they operate correctly.
 - Module Testing: A module encapsulates related components so can be tested without other system modules
 - Sub-system Testing: Testing on collection of modules should be concentrated on testing interface errors.
 - System Testing: It is concentrated on validating the functional or non-functional requirements
 - Acceptance Testing (Alpha): AT involves testing the system with data supplied by the system procurer rather than simulated data.

TECHNIQUES

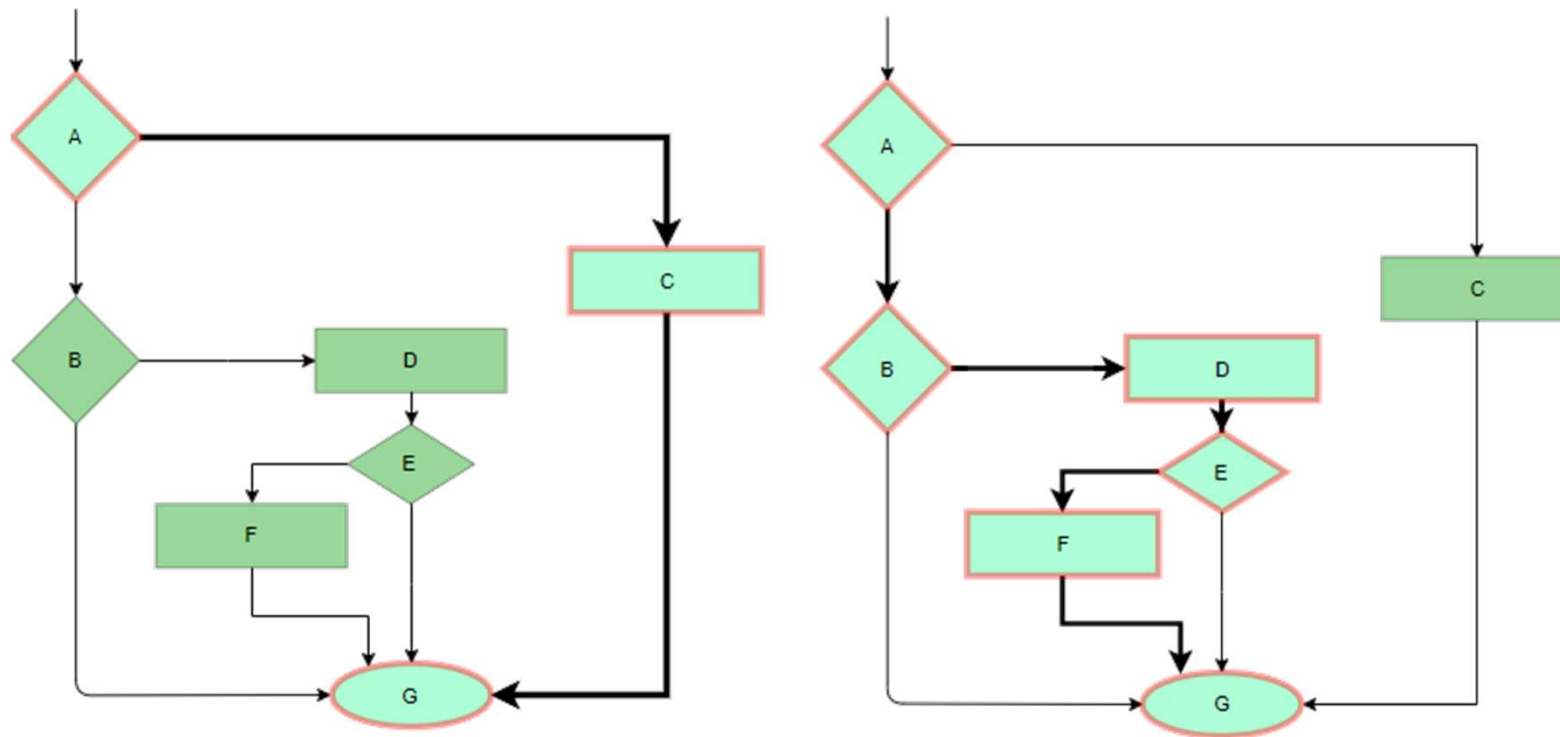
- Any engineered product can be tested in two ways;
 - Knowing the specified function and demonstrating fully works.
 - Knowing the internal workings and ensuring how it works

WHITE-BOX TESTING

- WB Testing is predicated on close examination of procedural detail.
- Flow
 - Draw a graph for conditions and data changes
 - Determine linearly independent paths
 - Prepare test cases for each path

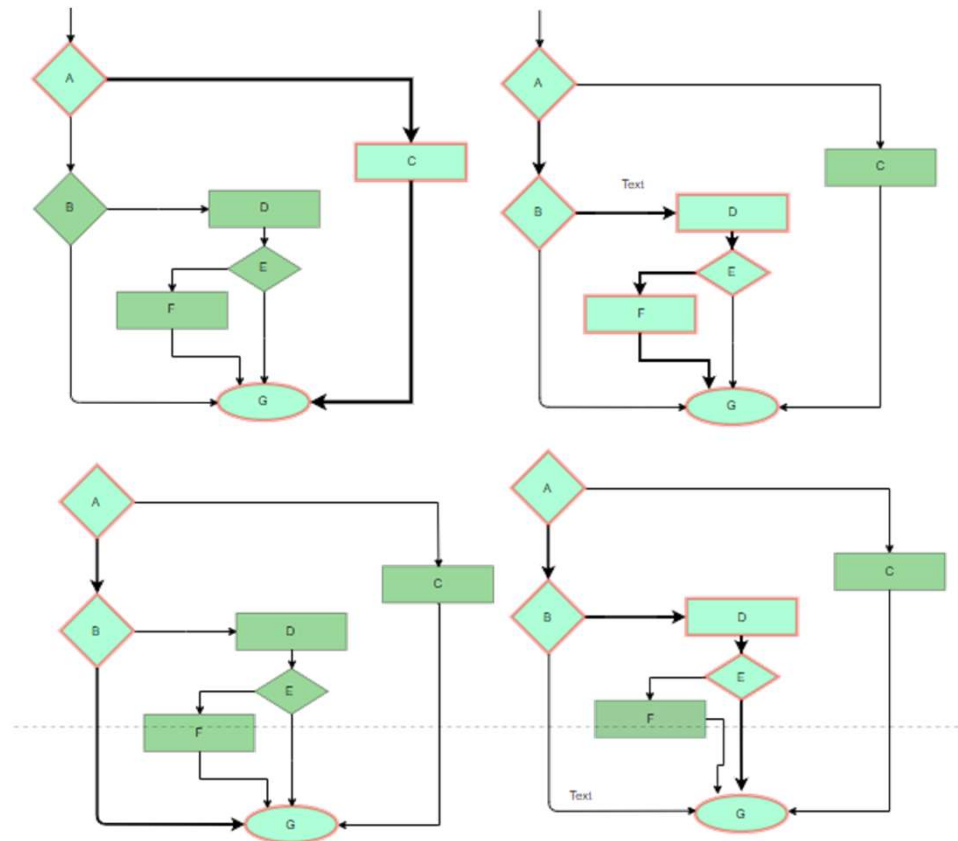
SAMPLE: STATEMENT COVERAGE

- Traverse all statements at least once...



SAMPLE: BRANCH COVERAGE

- Traverse each decision (from conditions) at least once



BLACK-BOX TESTING

- BB Testing enables the software engineer to derive sets of input conditions that will fully exercise all functional requirements for a program.
- Finding errors of; Incorrect or missing functions, Interface errors, Errors in data structures or external database access, Behaviour or performance errors, Initialization and termination errors.

SAMPLE: REQUIREMENT BASED

- Validate each requirement in the document.

SAMPLE: SYNTAX DRIVEN

- Test each grammar rule...

STRATEGIES

- Testing Strategies include;
 - Top-down
 - Bottom-Up
 - Thread
 - Stress

TOP-DOWN TESTING

- Top-down testing involves starting at the sub-system level to the modules.
- Unnoticed design errors may be detected at an early stage.
- Difficult to implement, because simulating lower levels of the system must be produced.

BOTTOM-UP TESTING

- Bottom-Up Testing involves testing the modules at the lower levels at first.
- Opposite of Top-Down Testing

THREAD TESTING

- Real Time Systems usually are cooperating processes
- 1.Test each process individually
- 2.Determine possible processing threads
- 3.Test each thread set with multiple inputs

STRESS TESTING

- Testing specified loads of the classes

RELIABILITY METRICS

- Reliability: How well software provides the services expected of it by its users
- Some metrics:
 - Probability of failure on demand
 - Rate of failure occurrence
 - Mean time to failure
 - Availability
- *No single metric is enough to determine.

STATISTICAL TESTING

- Principle objective of statistical testing is to determine the reliability of the software rather than to discover software faults.
- Steps:
 - Determine the operational profile of the software
 - Select or generate a set of test data
 - Apply these test cases
 - After a statistically significant number of failures have been observed...

TESTING TOOLS

- Instrumenting source code
- Running via a simulator
- Generating test data
- Predicting test data
- Comparing results

DYNAMIC ANALYSERS

- D.A.s are programs used to provide information on how often each statement in a program has been executed.

TEST DATA GENERATORS AND ORACLES

- T.D.G.s are programs which automatically generate a large number of test inputs for some system.
- Principle uses;
 - Performance of system
- Oracle is a program which, given a set of data, can predict what the output of the program test ought to be if there are no defects in the part of the system tested by that data.

FILE COMPARATORS

- F.C. is a general-purpose tool which reports differences between files.
- Principal uses;
 - Checking desired outputs
 - Checking many tests at a time
 - Checking variable initializations

SIMULATORS

- S. is a program which imitates the actions of some other program, hardware device or class of device.
- Principal uses;
 - To test Real-time systems
 - To test system under heavy-load

DEBUGGING ENVIRONMENTS

- D.E. involves locating and repairing program defects via monitoring the control and data flow of the program.
- Components;
 - Execution controller
 - Trace package
 - Symbol value recording package

TESTING - MGT

- Test Mgt is the practice of closely managing and monitoring application testing to ensure that resources are being focused on high-risk or important areas of the software.
- Basic Process
 - Planning
 - Execution

QUESTIONS?

CSE 3206–Software Engineering

Assoc. Prof. Dr. Kokten Ulas BIRANT

